Assignment: Inventory Management with Stock Validation

Duration: 1 Hour
Frontend: Vue 3
Backend: Java (Spring Boot) OR Golang
Database: MySQL OR PostgreSQL

Objective

Build a **Mini Inventory Management System** where:

- Admin can add products
- Users can view products
- Users can place orders
- Stock must reduce after order placement
- System must prevent ordering more than available stock

Section 1: Database Design (10 Marks)

1. Create the following tables:

Table 1: products

Column	Type	Constraints
id	Primary Key	Auto Increment
name	Varchar	Not Null
price	Decimal	Not Null
stock_quantity	Integer	Not Null
created_at	Timestamp	Default Current Time

Table 2: orders

Column	Type	Constraints
id	Primary Key	Auto Increment
product_id	Foreign Key → products(id)	Not Null
quantity	Integer	Not Null
total_price	Decimal	Not Null
created_at	Timestamp	Default Current Time

Section 2: Backend Development (40 Marks)

You must implement the following APIs:

A) Add Product

POST /api/products

Request:

```
{
 "name": "Laptop",
 "price": 50000,
 "stock_quantity": 10
}
```

Response:

- 201 Created
- Proper error handling

GET /api/products

Response:

```
[
 {
  "id": 1,
  "name": "Laptop",
  "price": 50000,
  "stock_quantity": 8
 }
]
```

C) Place Order (Critical Logic)

POST /api/orders

Request:

```
{
 "product_id": 1,
 "quantity": 2
}
```

Mandatory Business Rules

1. If stock_quantity < quantity
   - Return **400 Bad Request**
   - Message: "Insufficient stock"
2. On successful order:
   - Deduct stock
   - Insert record in orders
   - Calculate total_price = price × quantity
3. Must use **Database Transaction**
   - If any step fails → rollback

! Additional Requirements

nternal

- Proper status codes
- Proper error response format
- Validation (no negative quantity)
- Clean folder structure
- Business logic should NOT be inside controller

Section 3: Frontend (Vue 3) (25 Marks)

Create a simple UI with:

Page 1: Product Management

- Add product form
- Product listing table

Page 2: Place Order

- Select product
- Enter quantity
- Submit order
- Display success/error message

UI Expectations

- Clean layout
- API integration using Axios
- Basic validation
- Display updated stock after order

Section 4: Edge Case Handling (10 Marks)

System should handle:

- Ordering non-existing product
- Ordering zero/negative quantity
- Concurrent order attempt (basic handling awareness)

- Proper status codes
- Proper error response format
- Validation (no negative quantity)
- Clean folder structure
- Business logic should NOT be inside controller

Section 3: Frontend (Vue 3) (25 Marks)

Create a simple UI with:

Page 1: Product Management

- Add product form
- Product listing table

Page 2: Place Order

- Select product
- Enter quantity
- Submit order
- Display success/error message

UI Expectations

- Clean layout
- API integration using Axios
- Basic validation
- Display updated stock after order

Section 4: Edge Case Handling (10 Marks)

System should handle:

- Ordering non-existing product
- Ordering zero/negative quantity
- Concurrent order attempt (basic handling awareness)

- Invalid JSON body

Section 5: Code Quality & AI Usage (15 Marks)

Candidate must submit:

1. Project folder (Backend + Frontend)
2. README file including:
    - Setup instructions
    - AI tools used
    - Prompts used (copy paste)
    - Any debugging done manually

Bonus (Optional – 5 Marks Extra)

- Add DB index
- Add Swagger/OpenAPI